THE LONDON SCHOOL OF ECONOMICS AND POLITICAL SCIENCE

ST445 – Week 1
Managing and Visualising Data

Data, Data Types and Reproducible Workflows with Git and GitHub

Dr Leonardo Alves Dias



Agenda

- Introduction to ST445
- Introduction to Data
- Introduction to GitHub Workflows

Learning Outcomes

1. Critically evaluate how structured, semi-structured and unstructured data are represented, from primitive data types and the international standards that govern them for a given analytical problem.
4. Design a reproducible data preprocessing workflow on real-world data, diagnosing and remedying common data quality problems, and critically justifying each decision taken.

THE LONDON SCHOOL OF ECONOMICS AND POLITICAL SCIENCE

INTRODUCTION TO ST445

Timetable

Lectures:

Tuesdays
11:00 – 13:00

Usually recorded

Dr Leonardo Alves Dias

Seminar 1
Thursday: 11:00 – 12:30 (*Not recorded)* *Dr Leonardo Alves Dias*

Seminar 2
Thursday: 12:30 – 14:00 (*Not recorded)* *Dr Leonardo Alves Dias*

Seminar 3
Thursday: 17:00 – 18:30 (*Not recorded)* *Dr Kaifang Zhou*

Seminar 4
Friday: 13:30 – 15:00 (*Not recorded)* *Dr Kaifang Zhou*

Resources, Assignments, and Support

Course: ST445 - Managing and Visualising Data 2026/27 | Moodle





What is ST445 about?

- Principles and best practices for data manipulation and visualisation.
- Provide hands-on experience in carrying out a full data science project cycle, from data acquisition to reporting with conclusions and visualisations using Python.

- This course is NOT about state-of-the-art statistical modelling techniques/machine learning methods/artificial intelligence. Our department provides a range of other courses:
  - ST443 Machine Learning and Data Mining
  - ST447 Data Analysis and Statistical Methods
  - ST455 Reinforcement Learning
  - ST456 Deep Learning
  - ST457 Graph Data Analytics and Representation Learning
- Computer Programming course available (MY470).
- MY472 (Data for Data Scientists) has significant overlap with ST445.

Learning Outcomes

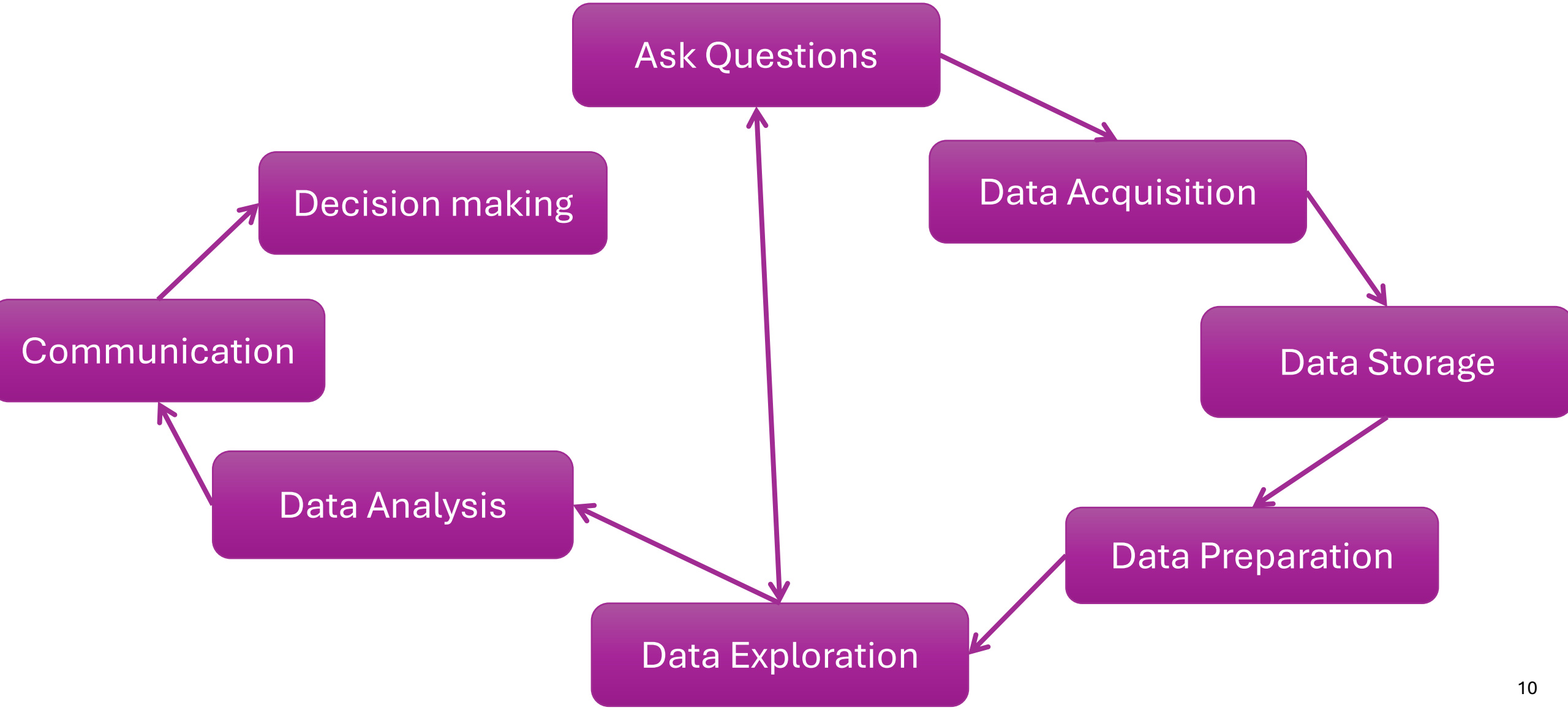
1. Critically evaluate how structured, semi-structured and unstructured data are represented, from primitive data types and the international standards that govern them, for a given analytical problem.
2. Implement relational and non-relational databases, carrying out create, read, update and delete (CRUD) operations using SQL and NoSQL.
3. Acquire data lawfully and ethically from files, REST APIs or web pages and integrate data from multiple sources through join operations on common keys.
4. Design a reproducible data preprocessing workflow on real-world data, diagnosing and remedying common data quality problems, and critically justifying each decision taken.
5. Conduct a systematic exploratory data analysis and critically interpret the results to extract, evaluate and evidence substantive insights.
6. Apply and evaluate visualisations grounded in the best practices and principles of human visual perception to communicate data-informed findings effectively to specialist and non-specialist stakeholders.

Who takes this course?

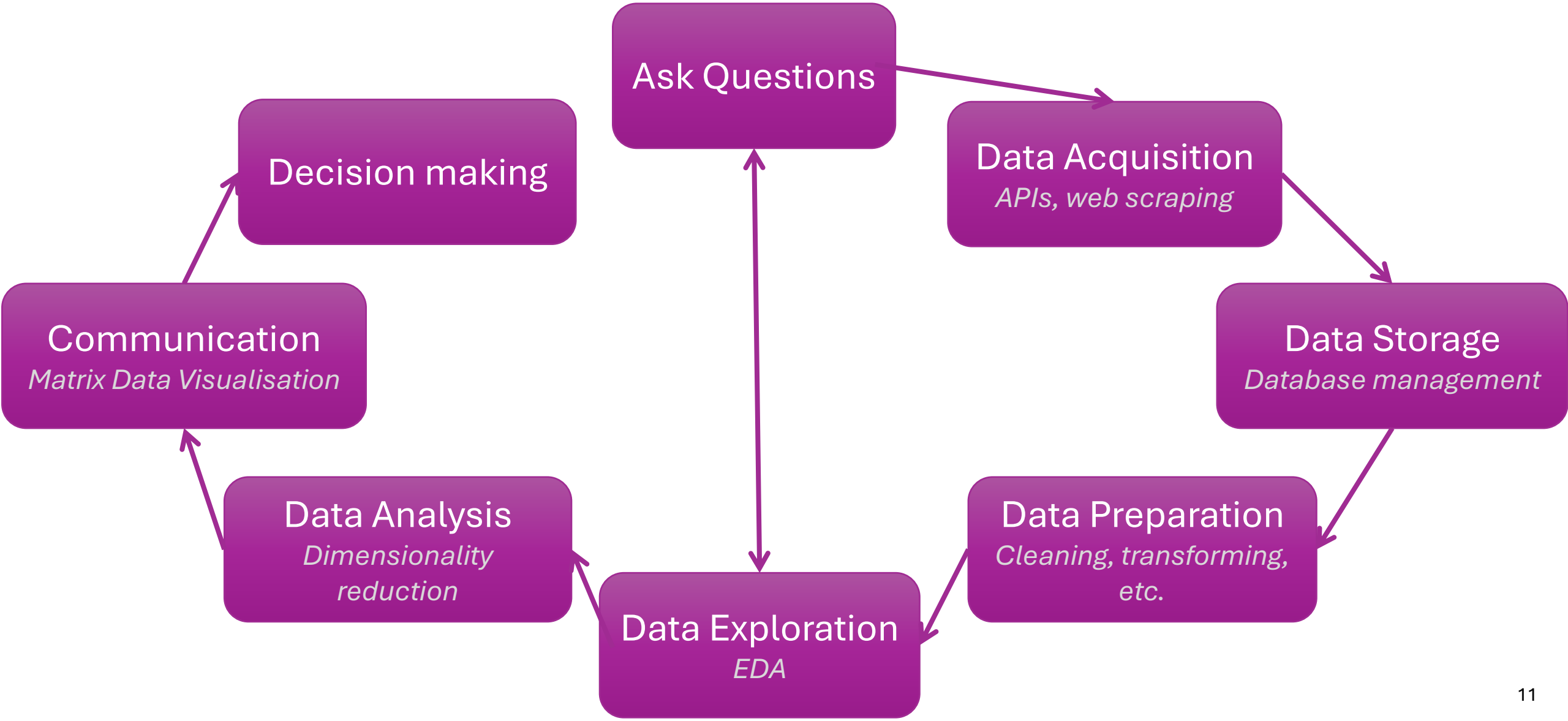
- Compulsory for MSc in Data Science and MSc in Health Data Science.

- Available for MPA in Data Science for Public Policy, MSc in Applied Social Data Science, MSc in Geographic Data Science, MSc in Statistics, MSc in Statistics (Financial Statistics), and others.

What is the data science project cycle?



What is the data science project cycle?



ST445 Topic schedule

Wk	Topic	Pipeline stage	LOs
1	Introduction to Data and Reproducible Workflows with GitHub	Foundations and version control	LO1, LO4, LO3
2	Collecting Data from the Web: Introduction to APIs and Web Scraping	Pipeline stage 1 — data collection	LO3, LO1
3	Representing Data: NumPy and pandas Data Structures	Pipeline stage 2 — representation	LO1, LO4
4	Data wrangling with pandas	Pipeline stage 3 — cleaning and preprocessing	LO4
5	Managing Data via Databases	Pipeline stage 4 — storage and query	LO2, LO3
6	Reading Week — no teaching	Consolidation	—
7	Exploratory Data Analysis and the Principles of Effective Visualisation	Pipeline stage 5 — exploration	LO5, LO6
8	Matrix Visualisation	Pipeline stage 5 — exploration continued	LO5, LO6
9	Visualising and Communicating Model Performance	Pipeline stage 5 — evaluation	LO5, LO6
10	Dimensionality Reduction for Visualisation	Pipeline stage 5 — high-dimensional exploration	LO5, LO6
11	Graph Data Visualisation and Communicating Data-Informed Decisions	Pipeline stage 6 — communication	LO6, LO5, LO2

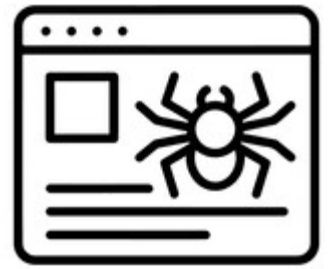
What will you get from the course?

1. Understand the main principles of data manipulation and visualisation.
2. Gain experience working with real-world data, tools, and techniques.
3. Capability to perform key steps in a data science project life cycle to address real-world problems .
4. Develop self-learning skills and critical thinking.

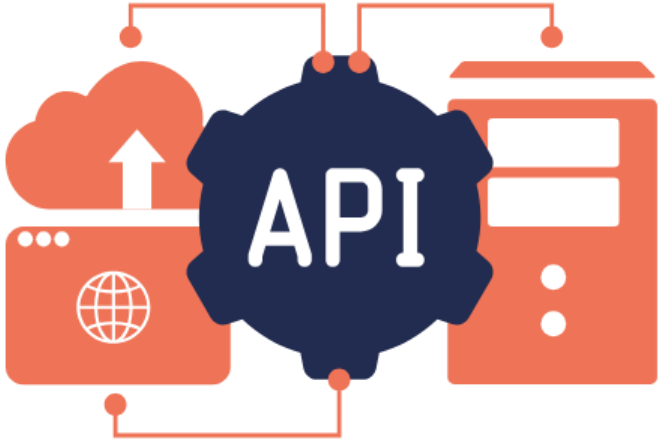
Data science tools used in the real world



colab



GitHub



THE LONDON SCHOOL OF ECONOMICS AND POLITICAL SCIENCE

INTRODUCTION TO DATA

37.9

What is this?

The same value but different context

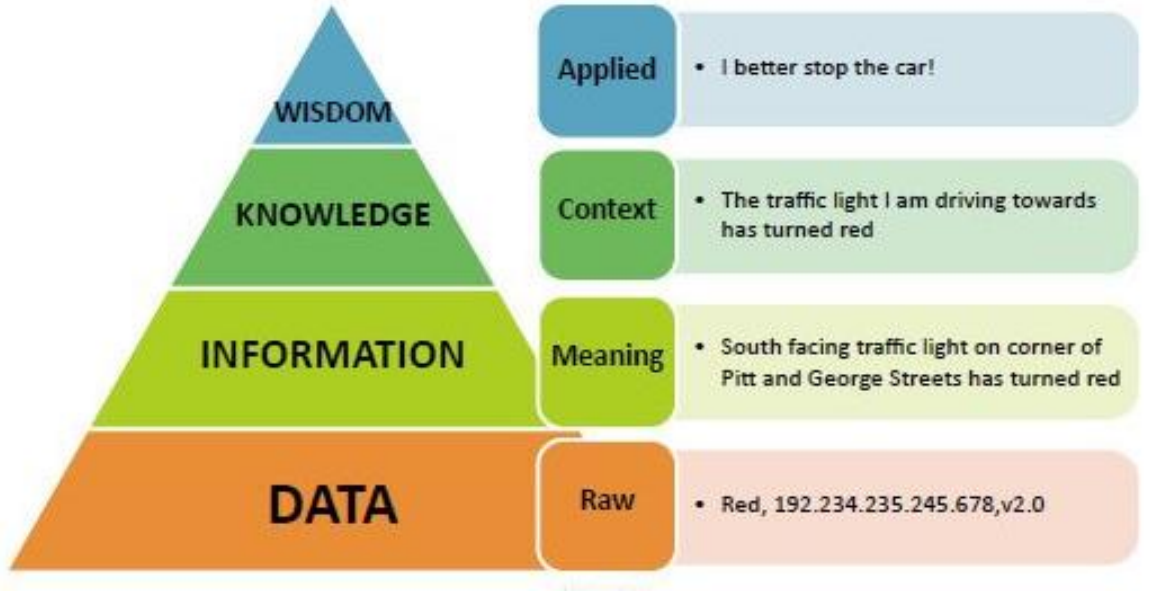
37.9

Unit	degrees Celsius
Measured	body temperature
Patient	4411
When	2026-10-05 08:15 UTC
Triage Category	2

Question it can now answer: Does this patient have a fever?

Data vs Information

- Data refers to raw, unorganised facts, numbers, symbols, or observations that have not yet been processed or given context (Singh, N., 2024).
- Meanwhile, information is when these facts and observations are analysed and presented in such a way that they are meaningful or can be used for decision-making.
- For example, a list of names is data. A register of who attended a class is information.



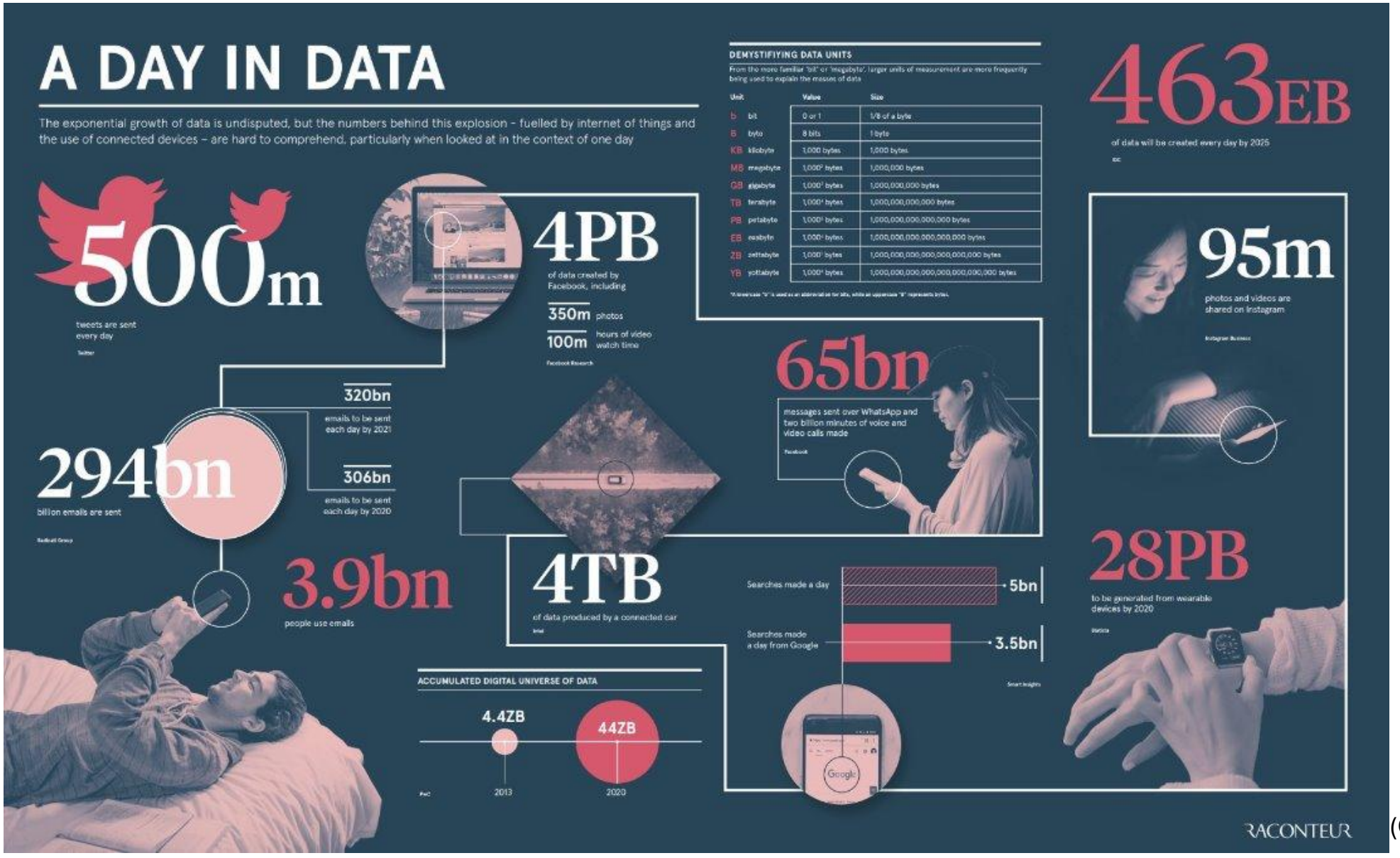
(Google, 2023)

Data becomes more and more complicated ...

Changes in the world of data

- High-dimensional data ($p \gg n$), e.g., genetic data (dimension reduction, penalised regression, random projection).
- Functional data, e.g., time series, images (functional principal component analysis, deep learning).
- Unstructured text data.
- Big data/massive data (subsampling, divide and conquer, parallel computing)
  - Volume of data in the modern world: In 2013, 90% of the world's data was generated from 2011 to 2013 (Science Daily, 2013).

Big Data Examples



(Google, 2023)

Generative AI









Behind Large Language Models (LLMs)

COMMON CRAWL

September 19, 2026

September 2026 Crawl Archive Now Available

We are pleased to announce the release of the September 2026 crawl archive, consisting of 2.17 billion pages (or 361.4 TiB of uncompressed content.) (Common Crawl, 2026)

1 TiB = 240 or 1,099,511,627,776 bytes

Common Crawl - Blog - September 2026 Crawl Archive Now Available

Behind Image Generation



(LAION, 2026)

What is Big Data?

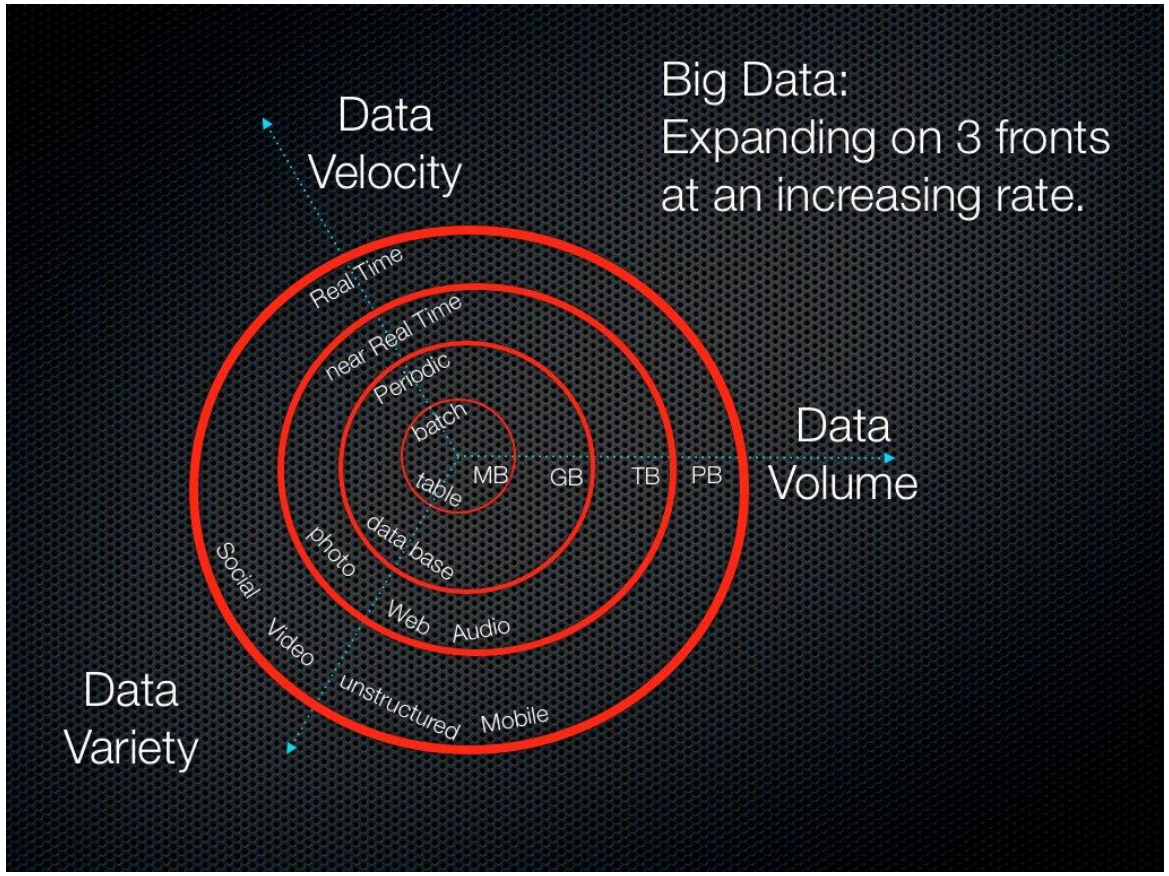






Big Data Definition

DATA that contains greater variety, arriving in increasing volumes and with more velocity.

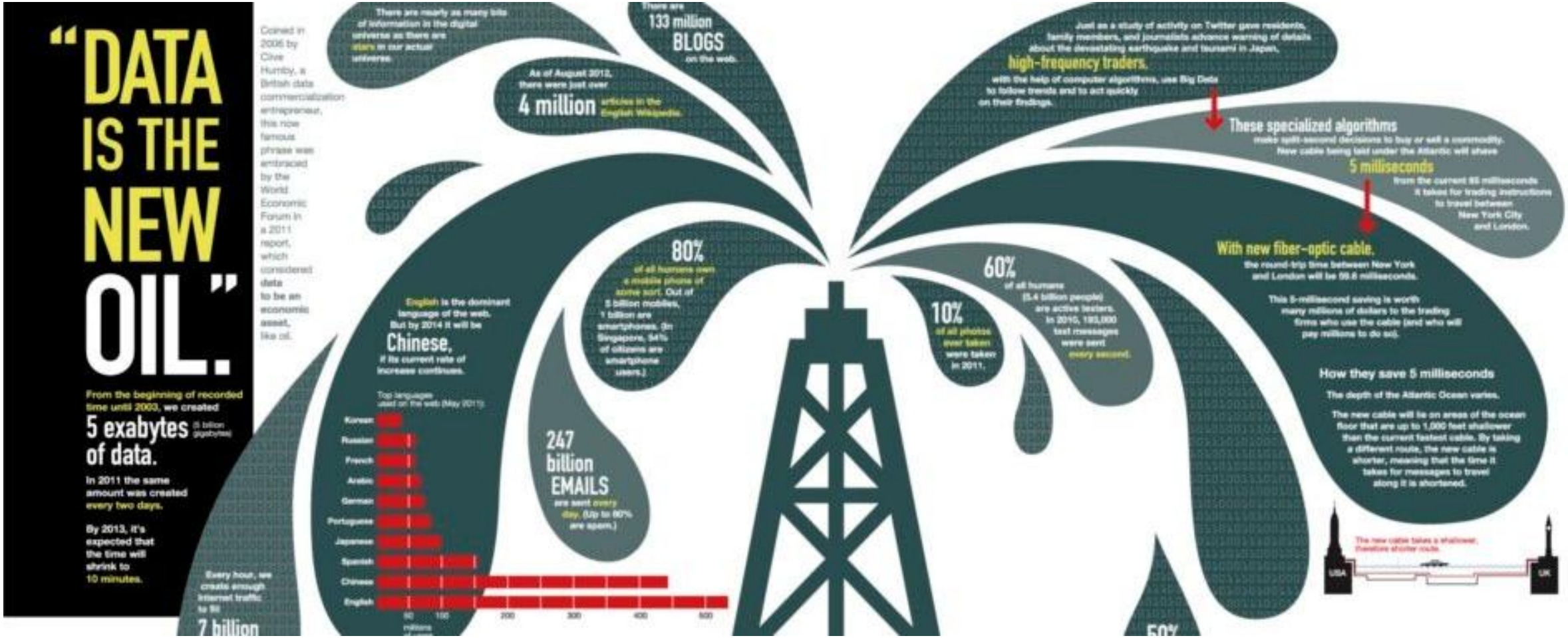


This is also known as the three “Vs”

(Oracle, 2024)

https://www.oracle.com/big-data/what-is-big-data

The value of data over time



Summary

- We defined the concepts of Data and Information
- We defined the concept of Big Data

STRUCTURE OF DATA

The Structure of Data

Data can be structured in different schemas

Structured

Fixed schema. Every record has the same typed fields.

Examples: Database tables, clean CSV, spreadsheets with one header row.

Semi-structured

Self-describing keys or tags. Records can differ and nest.

Examples: JSON, XML, HTML, GeoJSON, email headers.

Unstructured

No schema a machine can use without interpretation.

Example: Free text, images, audio, scanned PDFs.

Unstructured data

The university has 5600 students.
John's ID is number 1, he is 18 years old and already holds a B.Sc. degree.
David's ID is number 2, he is 31 years
Ph.D. deg
number 3
and also
degree a
degree.



Semi-structured data

```
<University>
  <Student ID="1">
    <Name>John</Name>
    <Age>18</Age>
    <Degree>B.Sc.</Degree>
  </Student>
  <Student ID="2">
    <Name>David</Name>
    <Age>31</Age>
    <Degree>Ph.D. </Degree>
  </Student>
  ....
</University>
```

Structured data

ID	Name	Age	Degree
1	John	18	B.Sc.
2	David	31	Ph.D.
3	Robert	51	Ph.D.
4	Rick	26	M.Sc.
5	Michael	19	B.Sc.

(Google, 2026)

More structure → less work before analysis

LSE THE LONDON SCHOOL OF ECONOMICS AND POLITICAL SCIENCE

Where you can find each kind

Health records

Admissions table with coded diagnoses

Structured

Public spending

Departmental spend over £25,000, published monthly

Structured

Survey responses

Likert items plus a free-text comments field

Mixed

Free text

Parliamentary debates, product reviews, social posts

Unstructured

Images

Satellite tiles, scanned forms, X-rays

Unstructured

Geospatial files

GeoJSON boundaries, shapefiles

Semi-structured

Real datasets can be a mix of all three

resp_id	q1_satisfied	q2_recommend	comments	device
R0012	4	5	Delivery was late but support fixed it fast	{"os": "iOS", "app": "3.2"}
R0013	2	1	never again!!!	{"os": "Android"}
R0014	5	5		{"os": "web", "ref": "email"}

Structured Unstructured text Semi-structured JSON inside a cell

Structured data is often easier to manage and prepare for analysis. The more unstructured the data, the more preparation it will likely need.

Primitive data types

Type	Example	Python	What can go wrong
Integer	42	int	Leading zeros stripped from IDs: 00731 → 731
Floating point	37.9	float	Large IDs lose digits; rounding surprises
Boolean	True	bool	"Yes", "Y", "1", "TRUE" all meaning the same
String	"SEPTIN2"	str	Encoding damage; text read as dates or numbers
Date and time	2026-10-05T08:15Z	datetime	Day and month swapped; time zone dropped
Missing		None, NaN	Many spellings, and real values mistaken for missing

How Python represents data

Why did YouTube have to rewrite its view counter in 2014?

```
2,147,483,647
```

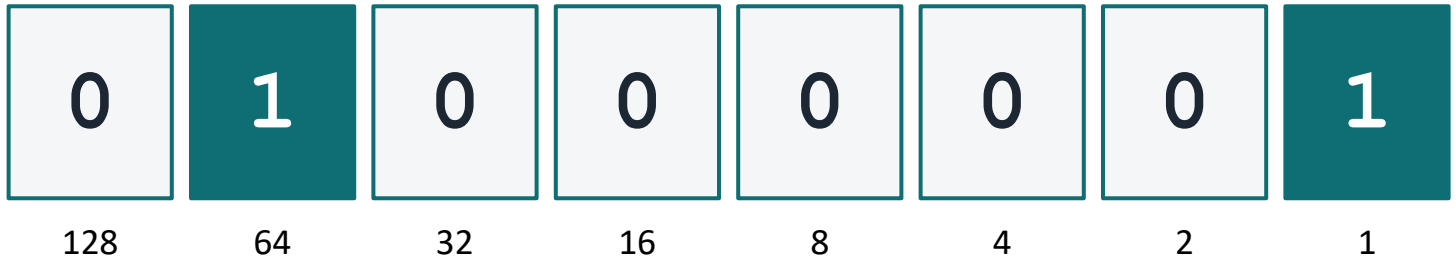
views, and the counter could not go one higher.



[Who, what, why: Why was Psy nearly too much for YouTube? - BBC News]

Everything is stored as bits

Data is stored as bits, and it can have different types. For instance, the value below can represent either the number *65* or the character *A*, depending on its type.



8 bits = 1 byte

```
>>> bin(65)
'0b1000001'
>>> chr(65), ord("A")
('A', 65)
```

Read as an integer

`65` 64 + 1

Read as a character

`A` code point 65

A type is a rule for reading bits. Same bits, different rule, different value.

LSE THE LONDON SCHOOL OF ECONOMICS AND POLITICAL SCIENCE

Fixed-width integers have hard limits

Type	Bits	Smallest	Largest
Unsigned 8-bit	8	0	255
Signed 8-bit	8	−128	127
Unsigned 32-bit	32	0	4,294,967,295
Signed 32-bit	**32**	**−2,147,483,648**	**2,147,483,647**
Signed 64-bit	64	-2^{63}	$2^{63} - 1 = 9{,}223{,}372{,}036{,}854{,}775{,}807$

n bits give 2^n patterns. Signed types spend half of them on negative numbers: -2^{n-1} to $2^{n-1} - 1$.

Python integers have no fixed limit

```
>>> 2**63 - 1
9223372036854775807

>>> 2**63 - 1 + 1          # no overflow
9223372036854775808

>>> x = 100**500
>>> len(str(x))             # digits
1001
```

Arbitrary precision

Python grows the number of digits it stores as the value grows.

The limit

Memory. A computer is finite, so it still holds only a finite set of integers.

NumPy integers are fixed width, and they wrap

```
>>> import numpy as np
>>> arr = np.array([2**32 - 1], dtype=np.uint32)
>>> arr
array([4294967295], dtype=uint32)

>>> arr + 1
array([0], dtype=uint32)              # wrapped round

>>> np.array([2**31 - 1], dtype=np.int32) + 1
array([-2147483648], dtype=int32)
```

No error is raised. The result is simply wrong.

Why 0?

`1111 … 1111`

the maximum: all 32 bits are 1

`+ 1`

`1 0000 … 0000`

the carry needs a 33rd bit

`0000 … 0000`

the 33rd bit is dropped: 0

Variable width or fixed width?

Python int: variable width

+ Can never overflow
– Slower and larger in memory

Good for: Scripts, counters, IDs, exact arithmetic

NumPy and pandas: fixed width

+ Fast, compact, vectorised
– Can overflow silently

Good for: Arrays and columns with millions of values

Ariane 5 (1996): A 64-bit value was converted to a 16-bit integer, but it didn't "fit". About 37 seconds after lift-off, the guidance system failed, and the rocket was destroyed.

The Ariane 5 Failure: How a Huge Disaster Paved the Way for Better Coding

Most real numbers are only approximated

```
>>> 6.14 - 3.14
2.9999999999999996

>>> 6.14 - 3.14 == 3
False

>>> import math
>>> math.isclose(6.14 - 3.14, 3)
True
```

Why

6.14 and 3.14 have no exact binary form, so each is stored as the nearest value that fits.

Rule

Never test floats with ==. Use math.isclose, or np.isclose for arrays.

LSE THE LONDON SCHOOL OF ECONOMICS AND POLITICAL SCIENCE

Characters are numbers too

Character	Code point	UTF-8 bytes	Bytes
A	65	41	1
é	233	C3 A9	2
€	8364	E2 82 AC	3
😀	128512	F0 9F 98 80	4

```
>>> ord("é")
233
>>> "é".encode("utf-8")
b'\xc3\xa9'
>>> len("café"),
len("café".encode("utf-8"))
(4, 5)
```

UTF-8 uses 1 to 4 bytes per character, so characters are not bytes.

LSE THE LONDON SCHOOL OF ECONOMICS AND POLITICAL SCIENCE

Python collections: holding many values

	list	tuple	dict	set
Written	`["cat", "dog"]`	`("cat", "dog")`	`{"GB": "GBP"}`	`{"GB", "DE"}`
Ordered	**Yes**	**Yes**	**Yes (insertion)**	No
Changeable	**Yes**	No	**Yes**	**Yes**
Duplicates	**Yes**	**Yes**	Keys unique	No
Access by	Position: x[0]	Position: x[0]	Key: x["GB"]	Membership: "GB" in x
Typical use	A sequence that grows	A fixed record	Lookup by key	Unique values

Choose based on what you need to do: grow it, fix it, look things up, or keep values unique.

Lists change; tuples do not

list

```
>>> pets = ["cat", "dog", "mouse"]
>>> pets[0] = "rabbit"
>>> pets.append("fish")
>>> pets
['rabbit', 'dog', 'mouse', 'fish']
```

tuple

```
>>> lse = (51.5144, -0.1165)
>>> lse[0] = 0
TypeError: 'tuple' object does
not support item assignment
```

Use a tuple when the values belong together and should not change.

Spot the bug

```
shopping = ["apples", "pears", "bananas"]
shopping.Sort()
print(shopping)
```

What happens when this runs, and how would you fix it?

Bonus: What does print(shopping.sort()) print?

Read the error message: it names the problem

```
>>> shopping.Sort()
AttributeError: 'list' object has no attribute 'Sort'

>>> shopping.sort()           # lower case: sorts in place
>>> shopping
['apples', 'bananas', 'pears']

>>> print(shopping.sort())    # in-place methods return None
None
>>> sorted(["pears", "apples"])   # returns a new list
['apples', 'pears']
```

Case matters

Sort and sort are different names.

In place returns None

.sort(), .append() and .reverse() change the list and return nothing.

New copy

sorted() returns a new list and leaves the original alone.

Primitive data types

Type	Example	Python	What can go wrong
Integer	42	int	Leading zeros stripped from IDs: 00731 → 731
Floating point	37.9	float	Large IDs lose digits; rounding surprises
Boolean	True	bool	"Yes", "Y", "1", "TRUE" all meaning the same
String	"SEPTIN2"	str	Encoding damage; text read as dates or numbers
Date and time	2026-10-05T08:15Z	datetime	Day and month swapped; time zone dropped
Missing		None, NaN	Many spellings, and real values mistaken for missing

Missing data has many spellings

(blank)	NA	N/A	NULL	None
-999	.	?	0	unknown

The red ones are also real values.

Decide what missing values look like in each source, convert them explicitly when you load the data, and record your decision.

LSE THE LONDON SCHOOL OF ECONOMICS AND POLITICAL SCIENCE

Standards remove the guessing

IEEE 754

Floating-point numbers

How is this number stored?

ISO 8601

Dates and times

Which moment is this?

Unicode and UTF-8

Text

Which characters are these bytes?

ISO 3166 and ISO 4217

Country and currency codes

Which country, which currency?

IEEE 754: floats are exact only so far

- The IEEE 754 is a technical standard for floating-point arithmetic to define how computers represent and calculate real numbers.

```
>>> 2**53
9007199254740992

>>> float(9007199254740993)
9007199254740992.0   # last digit lost
```

$$2^{53}$$

About 9×10^{15}. The largest integer a 64-bit float is guaranteed to hold exactly.

ISO 8601: one way to write a moment

ISO 8601 is the international standard for representing dates and times in a clear, unambiguous, and machine-readable way.

```
05/10/2026
```

5 October in the UK. 10 May in the US.

```
2026-10-05T14:30Z
```

date, year first

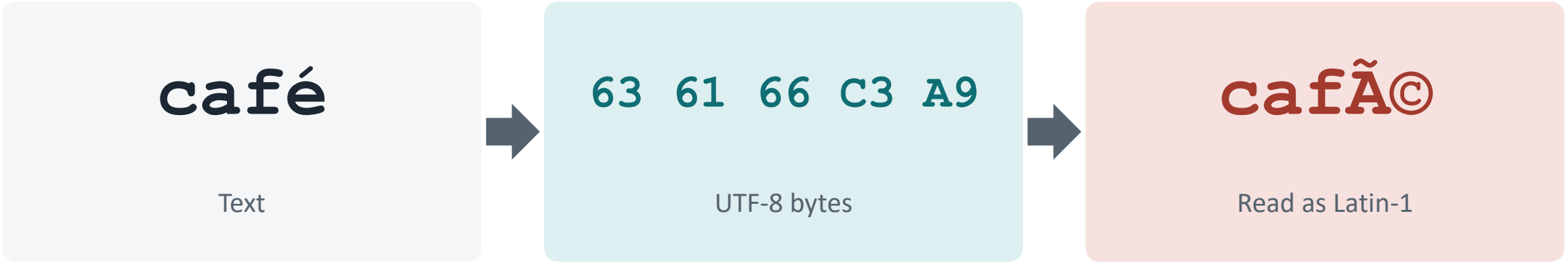
time

separator

UTC Time Format

Largest unit first, so it sorts correctly as text. Always keep the time zone.

Unicode and UTF-8: bytes need a decoder




```
>>> "café".encode("utf-8").decode("latin-1")
'cafÃ©'
```

Know the encoding of every file you read.

ISO 3166 and ISO 4217: codes are join keys

- ISO 3166 defines internationally recognised codes for country names, while ISO 4217 builds on it to define standard codes for global currencies

GB	United Kingdom
DE	Germany
BR	Brazil
NA	Namibia

GBP EUR BRL

ISO 4217 currency codes

```
>>> pd.read_csv("countries.csv")
  code      name
0   GB  United Kingdom
1   DE  Germany
2   BR  Brazil
3  NaN  Namibia

>>> pd.read_csv("countries.csv",
...              keep_default_na=False)
```

Many data failures happened because a tool guessed a type.

Standards exist so that nobody has to guess.

Higher-level data structures

Table

Records (rows) and fields (columns)

Weeks 3, 5

Key

A field that identifies a record and links tables

Week 5

Key–value pair

"ward": "B3"

Week 2

Key	Value
K1	AAA,BBB,CCC
K2	AAA,BBB
K3	AAA,DDD
K4	AAA,2,01/01/2015
K5	3,ZZZ,5623

(Wikipedia, 2026)

Nested document

Records inside records

Weeks 2, 5

Array

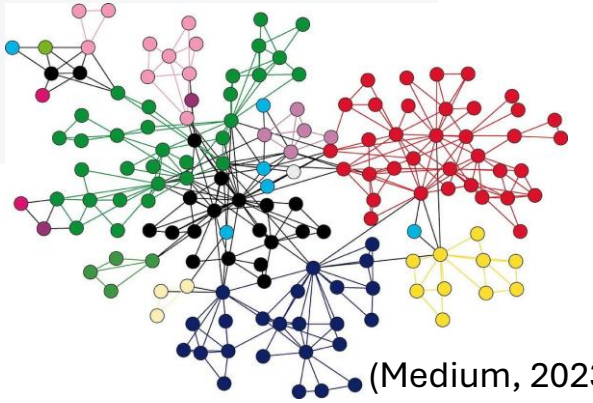
An ordered run of values of one type

Week 3

Graph

Nodes connected by edges

Week 11



(Medium, 2023)

Graph Data Structure in Python. In this article, you'll learn about... | Medium

Tidy data

Years spread across columns

borough	2023	2024	2025
Camden	412	398	405
Hackney	377	391	402



Tidy

borough	year	count
Camden	2023	412
Camden	2024	398
Camden	2025	405
Hackney	2023	377
…	…	…

Each variable is a column

Each observation is a row

Each type of unit is a table

Before you use any dataset, ask three questions

1

Where did it come from?

Provenance

Who made it, when, how, and what has changed since

2

What may I do with it?

Licensing

Publicly visible is not the same as free to reuse

3

Is anyone identifiable?

UK GDPR and ethics

Personal data needs a lawful basis; research may need ethics review

LSE THE LONDON SCHOOL OF ECONOMICS AND POLITICAL SCIENCE

Summary

- We defined the different structures of data
- We discussed some limitations of primitive data representations
- We briefly mentioned high-level data structures

THE LONDON SCHOOL OF ECONOMICS AND POLITICAL SCIENCE

GITHUB: REPRODUCIBLE WORKFLOWS

Introduction to GitHub

GitHub is a cloud-based platform that helps people store, manage, and collaborate on software projects using **Git**, a version control system.

With GitHub, you can:

- Store your code and project files in online repositories
- Track changes over time and view the history of a project
- Collaborate with others without overwriting each other's work
- Work on separate branches and merge changes when ready
- Manage issues, tasks, and project documentation
- Publish and share projects with others

Why are we using GitHub?

In this module, GitHub will help you:

- Keep track of changes to your work
- Collaborate effectively with team members
- Experiment safely using branches
- Develop industry-standard skills used in software development, data science, and research

Think of GitHub as Google Docs for code and projects, but with powerful tools for tracking, reviewing, and managing changes.

Reproducibility is a professional standard

Same data + same code + same environment = same result, for anyone

You, in six months

Cannot remember which cleaning step dropped 400 rows, or why.

Your collaborator

Cannot run your notebook: different paths, packages, data.

Your reader or marker

Cannot check the claim, so cannot trust it.

Undocumented analysis is not a finished analysis.

GitHub Key Terms

- **Repository (Repo):** A project's folder on GitHub
- **Commit:** A saved version of your work
- **Branch:** An independent line of development
- **Merge:** Combining changes from different branches
- **Pull Request (PR):** A request to review and merge changes

Git is the tool that tracks changes; GitHub is the platform that hosts and facilitates collaboration using Git.

What version control gives you

A full history

Every change, with who made it, when, and why

A way back

Return to any earlier version of any file

Parallel work

Try an idea on a branch without breaking what works

A shared copy

Collaborators work on the same project without emailing files

The Git model: three places a change can be

Working directory

Your files as they are on disk now

`git add` →

Staging area

Changes chosen for the next commit

`git commit` →

Repository

The history of commits

```
git status          # where is each change?
```

Run it constantly.

An example: your first commit

```
git clone https://github.com/<you>/st445-week1-demo.git
cd st445-week1-demo
git status              # working tree clean

# edit README.md, save
git status              # modified: README.md   (not staged)

git add README.md
git status              # changes to be committed: README.md

git commit -m "Add data source and licence to README"
git log --oneline
```

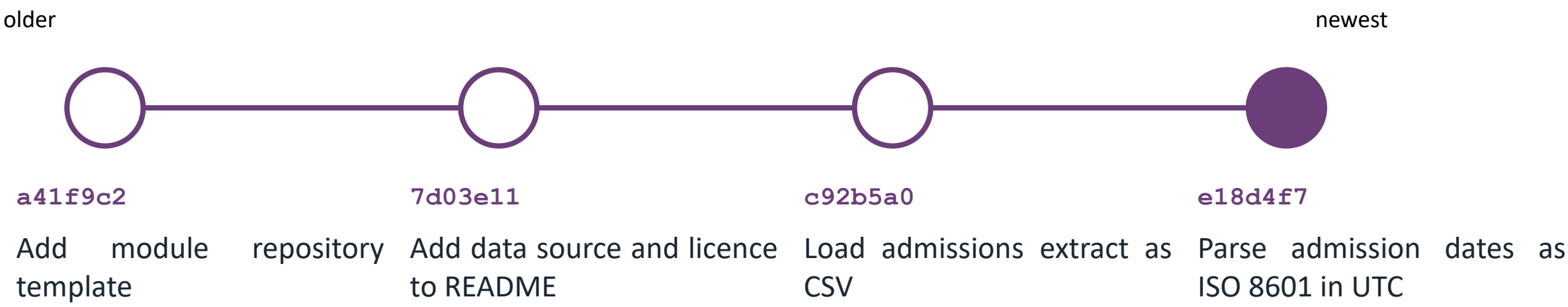
Watch the change move

Working directory

Staging area

Repository

A commit is a snapshot with a message



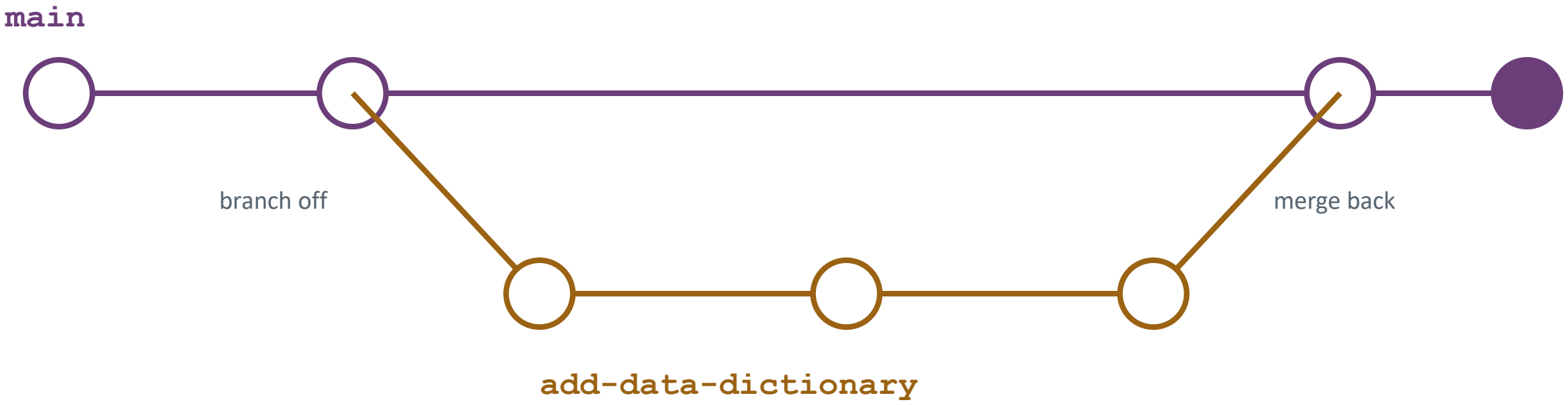
Each dot records every file: who, when, and why. You can return to any of them.

A useful commit message

Tells you nothing	Tells you what and why
`update`	`Parse admission dates as ISO 8601 in UTC`
`fix`	`Drop 14 duplicate patient rows before join`
`stuff`	`Read country codes with keep_default_na=False`
`asdfgh`	`Add data dictionary for admissions extract`
`final version`	`Move raw data to data/raw and make read-only`

Short summary in the imperative: what changed and why, if not obvious. One change per commit.

Branches: try something without breaking main




```
git switch -c add-data-dictionary     # create the branch and move onto it
```

Your laptop and GitHub

Your laptop

Local repository: working directory, staging area and full history

`git clone`

copy the remote once

`git push`

send your commits

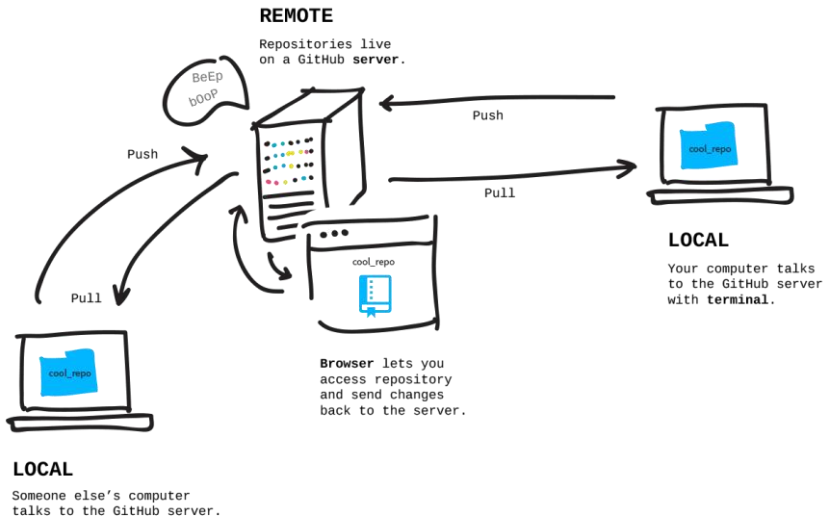
`git pull`

fetch + merge theirs

GitHub

Remote repository, called origin. What your partner sees.

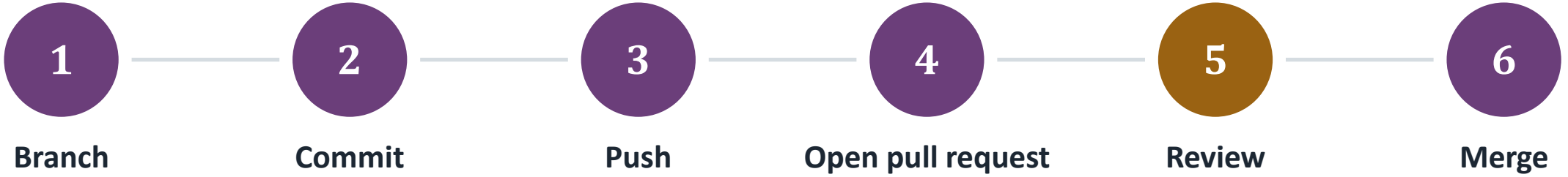
- Nothing reaches GitHub until you push.



GitHub for collaboration

README	The front door: what this is, how to run it, where the data came from
`.gitignore`	Files Git should never track
Issues	A to-do list attached to the code
Pull requests	A proposal to merge a branch, open for discussion
Code review	Comments on specific lines before anything merges

The pull request workflow




```
git push -u origin add-data-dictionary
# then on GitHub: Compare & pull request
```

Be specific about the code and suggest a fix.

A useful review comment

"Line 12 reads dates without a time zone. Could we parse them as UTC so that this matches the API data?"

What never goes into the repository

```
# .gitignore

# secrets
.env

# large or private data
data/raw/
*.parquet

# generated files
__pycache__/
.ipynb_checkpoints/
.venv/
.DS_Store
```

Secrets

A pushed API key should be treated as compromised, even after you delete it.

Large or private data

Keep it out; the README says how to get it.

Generated files

Rebuilt by running the code, so never stored.

A project a stranger can run

```
st445-project/
├── README.md
├── .gitignore
├── pyproject.toml   uv.lock
│   (or environment.yml)
├── data/
│   ├── raw/           never edited
│   └── processed/     rebuilt by code
├── notebooks/
├── src/
└── outputs/figures/
```

Capture the environment

uv

```
uv add pandas pyarrow
uv sync           # rebuild it anywhere
```

conda

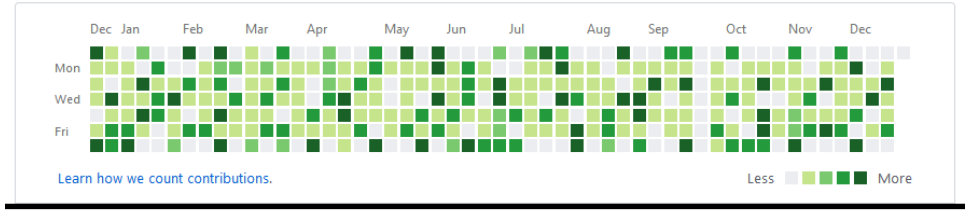
```
conda env export --from-history \
  > environment.yml
```

Same code, different package versions, different results.

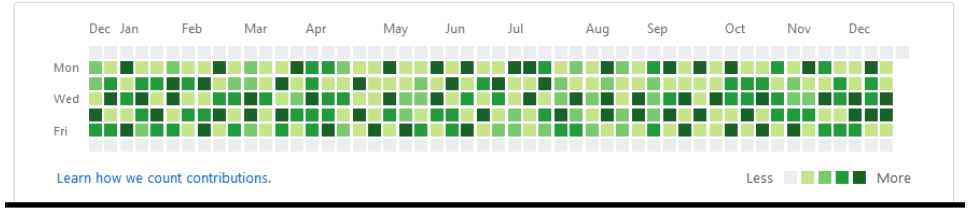
Resources for using git/GitHub

- An easy git Cheatsheet, by Nina Jaeschke and Roger Dudler
- git - the simple guide by Roger Dudler
- Free stuff for students! https://education.github.com/pack

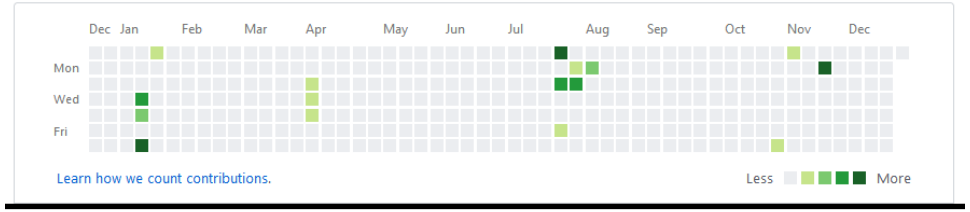
"The Open Source Enthusiast"



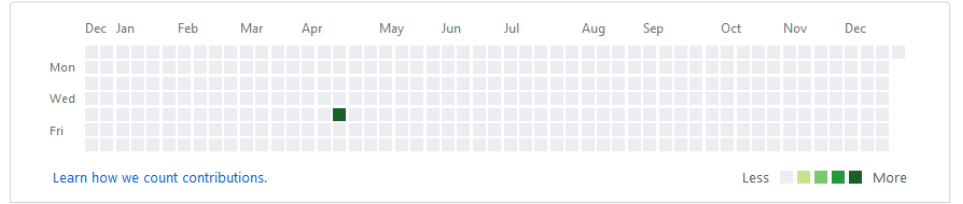
"My company uses github"



"Let's learn programming. For real this time"



"The curious"




When your shower uses github more than you

Summary



We introduced GitHub.

References

- Pedhazur, Elazar J.; Schmelkin, Leora and Albert (1991). *Measurement, Design, and Analysis: An Integrated Approach* (1st ed.). Hillsdale, NJ: Lawrence Erlbaum Associates. pp. 15–29. ISBN 978-0-8058-1063-9.